

Math analysis : ดูค่าฝั่งซ้ายแทน

ex. for (k=n; k>1; k--) {

for (i=2; i<=k; i++) {
command

$$\text{จำนวน} = \sum_{k=2}^n \left(\sum_{i=2}^k 1 \right) = \sum_{k=2}^n (k-1) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$$

★ speed : ดูที่ค่า process ฝั่ง / จำนวนหน่วย

Growth Rate f(n) vs g(n)

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \text{f(n) โตช้ากว่า} \\ c > 0 & \text{โตเท่ากัน} \\ \infty & \text{f(n) โตเร็วกว่า} \end{cases}$$

★ โตช้ากว่า = ดีกว่า

Asymptotic Notation : ใช้ e

little-o (o) : โตช้ากว่า
little-omega (ω) : โตเร็วกว่า
Big-O (O) : โตช้ากว่า/เท่ากัน : ขอบบน
Big-Omega (Ω) : โตเร็วกว่า/เท่ากัน : ขอบล่าง
Big-Theta (Θ) : โตเท่ากัน : ขอบเขต

Case

- Worst-Case : ทำการหาค่าสูงสุด
- Best-Case : ทำการหาค่าต่ำสุด
- Avg-Case : การทำงานโดยเฉลี่ย

Worst : O(n)
Best : O(1)
Avg : O(n)

Insertion sort : แบ่ง sorted / unsorted ตามลำดับ

↳ เอาตัวหน้า sorted แยก tmp กับ

	i	1	2	3	4	temp
Data		30	54	27	70	
first		30	54	27	70	54
second		27	30	54	70	27
third		27	30	54	70	70

Shell sort : แบ่งกลุ่มตาม round [n/2]

↳ sort กลุ่มละ round = 1

ex.

n=7	Group	0	1	2	3	4	5	6
D=round[7/2]=4		10	25	35	73	25	23	27
sorted result		10	27	73	25	23	35	97

★ ทำจน Group = 1 = sort เสร็จ

```
int partition(int A[], int p, int r) {
    int c = A[p]; // เลือกตัวแรกเป็น Pivot(c)
    int i = p - 1; // ตัวนี้ฝั่งซ้าย
    int j = r + 1; // ตัวนี้ฝั่งขวา

    while (true) {
        // ร้องด้วย j จากขวาซ้าย เพื่อหาตัวที่ <= Pivot
        do {
            j--;
        } while (A[j] > c);

        // ร้องด้วย i จากซ้ายไปขวา เพื่อหาตัวที่ >= Pivot
        do {
            i++;
        } while (A[i] < c);

        // ถ้าตัวนี้ยังไม่จบกัน ให้สลับค่า
        if (i < j) {
            int tmp = A[i];
            A[i] = A[j];
            A[j] = tmp;
        } else {
            // ถ้าจบกันแล้ว ให้คืนค่าตำแหน่ง j เพื่อใช้แบ่งกลุ่ม
            return j;
        }
    }
}
```

Binary Search : sorted → แบ่งครึ่งซ้ำๆ

Best: O(1), Worst: O(logn), Avg: O(logn)
ex. find 60 from

low	10	20	30	40	50	60	70	high
				low	mid	mid	high	
					60	60	70	

Time Complexity

★ เทียบ f(n) ที่ช้ากว่าเวลาการสุ่ม

- if... then... else : True/False เลือกอันที่ช้าที่สุด 1
- For loop : ให้ดูตาม sigma

ex. for (k=n; k>1; k--) {
for (i=2; i<=k; i++) {
command

$$\sum_{j=1}^n \sum_{i=1}^j \Theta(1) = \sum_{j=1}^n \frac{j+1}{2} \Theta(1) = \Theta(n^2)$$

void insertionSort(int data[], int n) {
// เริ่มจากตัวที่ 2 เพราะตัวที่ 1 index 0 เรียงเสร็จแล้ว
for (int i = 1; i < n; i++) {

```
int tmp = data[i]; // tmp = ค่าที่เราจะสลับไปหาที่ว่าง
int j = i - 1; // j = ตำแหน่งสุดท้ายของกลุ่มที่เรียงแล้ว

// วนลูปก่อนเพื่อเปรียบเทียบ 'tmp' กับกลุ่มที่เรียงแล้วทางซ้าย
// ถ้าตัวซ้ายมือ (data[j]) ยังมากกว่า 'tmp'
// ให้ "ขยับ" ตัวนั้นไปทางขวา 1 ช่อง เพื่อเปิดพื้นที่ว่าง
while (j >= 0 && data[j] > tmp) {
    data[j + 1] = data[j]; // ขยับข้อมูลไปทางขวา
    j = j - 1; // ลด j ไปดูตัวก่อนหน้า
}
```

// เมื่อเจอตัวที่ที่เหมาะสมแล้ว (หรือถึงขอบซ้ายสุด) ให้วาง 'tmp' ลงไป
data[j + 1] = tmp;

void shellSort(int data[], int n) {
// 1. กำหนดระยะห่าง (Gap หรือ D) เริ่มต้นที่ n/2
// และลดลงทีละครึ่งในแต่ละรอบจนกระทั่ง D = 1
for (int D = n / 2; D > 0; D /= 2) {

```
// 2. ทำการเรียงแบบ Insertion Sort
// สำหรับกลุ่มข้อมูลที่มีขนาดเท่ากับ D
for (int i = D; i < n; i++) {
    // ออกค่าที่กําลังจะตรวจสอบ
    int tmp = data[i];
    int j = i;

    // 3. เปรียบเทียบและขยับข้อมูลที่มีค่ามากกว่าเป็นระยะ D
    // ถ้าตัวก่อนหน้า (D ช่อง) มีค่ามากกว่าตัวปัจจุบัน
    // ให้ขยับตัวนั้นมาทางขวา
    while (j >= 0 && data[j - D] > tmp) {
        data[j] = data[j - D]; // ขยับข้อมูล
        // ลดค่า j ไปอีก D ตัวเพื่อไปหาตำแหน่ง D ช่อง
        j -= D;
    }
    data[j + D] = tmp;
}
```

// 4. วางค่าที่เก็บไว้ลงในตำแหน่งที่ถูกต้อง
data[j] = tmp;

```
void quickSort(int A[], int p, int r) {
    // เงื่อนไขหยุด: ถ้าตำแหน่งเริ่มต้น(p) ยังน้อยกว่าตำแหน่งสุดท้าย(r)
    if (p < r) {
        // 1. Partition: แบ่งข้อมูลและหาตำแหน่ง (j)
        int j = partition(A, p, r);

        // 2. Recursion: ทำซ้ำในฝั่งซ้าย (p ถึง j)
        quickSort(A, p, j);

        // 3. Recursion: ทำซ้ำในฝั่งขวา (j+1 ถึง r)
        quickSort(A, j + 1, r);
    }
}
```

int binarySearchIterative(int A[], int n, int x) {
int left = 0;
int right = n - 1;

```
// วนลูปจนกว่าจะเจอการค้นหาค่าในขอบเขต
while (left <= right) {
    // คำนวณหาจุดกึ่งกลางในแต่ละรอบ
    int m = left + (right - left) / 2;

    // กรณีที่ 1: เจอข้อมูลที่ถูกต้อง
    if (A[m] == x) {
        return m;
    }

    // กรณีที่ 2: ค่าที่หามาจะน้อยกว่าค่ากลาง ให้เลื่อนขอบขวา
    if (x < A[m]) {
        right = m - 1;
    }

    // กรณีที่ 3: ค่าที่หามาจะมากกว่าค่ากลาง ให้เลื่อนขอบซ้ายไป
    else {
        left = m + 1;
    }
}
```

// ถ้าหลุดขอบออกมาให้แสดงว่าไม่เจอ
return -1;

Sorting

Bubble Sort : Compare swap เปรียบเทียบ

↳ หากสุดท้ายจะสลับไปหลังสุดเรื่อยๆ

```
void bubbleSort(int data[], int n) {
    // กำหนดตัว swap เพื่อเช็คว่ามีสลับหรือไม่
    int swapped = 1;
    // วนลูปจนกว่าจะไม่มีสลับเกิดขึ้น (ยังเรียงไม่เสร็จ)
    while (swapped == 1) {
        // ตั้ง 0 ก่อนเริ่มตรวจสอบในแต่ละรอบ
        swapped = 0;

        // วนลูปเปรียบเทียบข้อมูลที่อยู่ติดกัน
        for (int i = 0; i < n - 1; i++) {
            // ถ้าตัวซ้ายมากกว่าตัวขวา ให้สลับกัน
            if (data[i] > data[i + 1]) {
                // ขั้นตอน Swap (สลับที่)
                int tmp = data[i];
                data[i] = data[i + 1];
                data[i + 1] = tmp;
                // เมื่อสลับ ให้เปลี่ยนค่าเป็น 1
                swapped = 1;
            }
        }
    }
}
```

Worst : O(n)
Best : O(1)
Avg : O(n)

All case : O(n²)

Selection Sort : หา min swap ตามลำดับ

ex. data : 5 1 4 2 3
first : 1 5 4 2 3
second : 1 2 4 5 3
third : 1 2 4 5 3
fourth : 1 2 4 5 3

```
void selectionSort(int data[], int n) {
    // Outer loop: วนรอบทั้งหมด n-1 รอบ
    for (int i = 0; i < n - 1; i++) {
        // 1. สมมติให้ตำแหน่งปัจจุบัน (i) คือค่าที่น้อยที่สุด (min_idx)
        int min_idx = i;

        // 2. Inner loop: ค้นหาตัวที่น้อยที่สุดจริงในส่วนของที่เหลือ (i+1 ถึง n)
        for (int j = i + 1; j < n; j++) {
            // ถ้าเจอตัวที่น้อยกว่าค่า min_idx ปัจจุบัน
            if (data[j] < data[min_idx]) {
                min_idx = j; // เก็บตำแหน่งใหม่ที่มีน้อยกว่า i
            }
        }

        // 3. เมื่อเจอตัวที่น้อยที่สุดในรอบนั้นแล้ว จึงทำการ Swap
        // การสลับจะเกิดขึ้นบนข้อมูลใน memory เพื่อ Reduce Swap
        if (min_idx != i) {
            int tmp = data[i];
            data[i] = data[min_idx];
            data[min_idx] = tmp;
        }
    }
}
```

Divide & Conquer

Merge Sort : แบ่งครึ่งไปเรื่อยๆ แล้ว merge Merge O(n)

↳ merge like reverse, sort ไปเรื่อยๆ

Worst = Avg = Best = 2T(n/2) + O(n) ∈ O(n log n)

```
void merge(int A[], int p, int q, int r) {
    int n1 = q - p + 1; // จำนวนสมาชิกฝั่งซ้าย
    int n2 = r - q; // จำนวนสมาชิกฝั่งขวา
```

```
// สร้างอาร์เรย์ชั่วคราว L และ R
int L[n1 + 1], R[n2 + 1];
```

```
// คัดลอกข้อมูลลงในอาร์เรย์ชั่วคราว
for (int i = 0; i < n1; i++) L[i] = A[p + i];
for (int j = 0; j < n2; j++) R[j] = A[q + 1 + j];
```

```
// ใส่ค่า Infinity ไว้ที่ท้ายอาร์เรย์เพื่อใช้เป็นตัวสิ้นสุด (Sentinel)
L[n1] = 2147483647; // สมมติว่าเป็นค่าสูงสุดของ int
R[n2] = 2147483647;
```

```
int i = 0, j = 0;
// วนลูปเพื่อเลือกตัวที่น้อยที่สุดจาก L และ R กลับเข้าไปใน A
for (int k = p; k <= r; k++) {
    if (L[i] <= R[j]) {
        A[k] = L[i];
        i++;
    } else {
        A[k] = R[j];
        j++;
    }
}
```

```
void mergeSort(int A[], int p, int r) {
    if (p < r) {
        // 1. Divide: หาจุดกึ่งกลาง
        int q = (p + r) / 2;

        // 2. Conquer: ส่งเรียงข้อมูลฝั่งซ้ายและขวา (Recur)
        mergeSort(A, p, q);
        mergeSort(A, q + 1, r);

        // 3. Combine: นำข้อมูลที่เรียงแล้วทั้งสองฝั่งมารวมกัน
        merge(A, p, q, r);
    }
}
```

Quick Sort

step: 1. เลือก pivot (โดยปกติ)

2. แบ่งข้อมูลรอบๆ pivot ซ้ายมากกว่าขวา

```
ex. 27 43 3 9 32 10
pivot 10
swap 10 27 43 3 9 32 10
swap 10 27 9 3 9 32 10
swap 10 27 9 3 32 10
3 27 9 10 43 32
3 9 27 10 43 32
Sorted 3 9 10 27
pivot 3
3 9 10 27 43 32
```

T(n) → Best: O(n log n), Avg: O(n log n), Worst: O(n²)

```
int binarySearchRecursive(int A[], int left, int right, int x) {
    // 1. Base Case: ถ้าขอบซ้ายรวมกับขอบขวา แสดงว่าหาไม่เจอ
    if (left > right) return -1;

    // 2. หาจุดกึ่งกลาง (Midpoint)
    int m = (left + right) / 2;

    // 3. ตรวจสอบเงื่อนไข
    if (x == A[m]) return m; // เจอแล้ว! คืนค่าตำแหน่ง

    if (x < A[m]) {
        // ค้นหาในฝั่งซ้าย (ตัดฝั่งขวาทิ้ง)
        return binarySearchRecursive(A, left, m - 1, x);
    } else {
        // ค้นหาในฝั่งขวา (ตัดฝั่งซ้ายทิ้ง)
        return binarySearchRecursive(A, m + 1, right, x);
    }
}
```

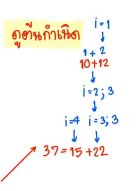

Dynamic Programming

0-1 Knapsack problem $\rightarrow T(n) = O(n \times W)$

capacity j (w=5)

weigh	value	i	1	2	3	4	5
W1=2	V1=12	1	0	12	12	12	12
W2=1	V2=10	2	10	12	22	22	22
W3=3	V3=20	3	10	12	22	30	32
W4=2	V4=15	4	10	15	25	30	37

Selected = 1, 2, 4



All pair shortest path $\rightarrow$ พิจารณาจุดที่ k

$D_{ij}^{(k)} = \min(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)})$

step by step:

- 1. เริ่มต้น $D^{(0)}$ ถ้า $i=j$ (ที่ค่าเป็น 0) มีเส้นที่เชื่อม weight = 0
- 2. Loop k ตั้งแต่ 1 ถึง n
- 3. ทำการหาทุก k ตารางสุดท้ายคือคำตอบ

```
// n: จำนวนโหนดในกราฟ
// w: Weight Matrix (ตารางน้ำหนัก) โดยที่:
// - w[i][j] = น้ำหนักของเส้นเชื่อม
// - w[i][i] = 0
// - w[i][j] = INF (หาไปไม่ได้เส้นเชื่อมตรง)

void AllPairShortestPath(int n, vector<vector<int>>& w) {
    // 1. กำหนดค่าเริ่มต้น: ตารางค่า d และค่าของค่า w
    vector<vector<int>> d = w;

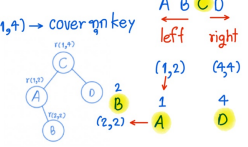
    // 2. Loop นอกสุด: เลือกจุดแวะพัก k ตั้งแต่ 0 ถึง n-1
    for (int k = 0; k < n; k++) {
        // 3. Loop กลาง: เลือกจุดเริ่มต้น i
        for (int i = 0; i < n; i++) {
            // 4. Loop ในสุด: เลือกจุดปลายทาง j
            for (int j = 0; j < n; j++) {
                // ตรวจสอบว่ามีเส้นทางจาก i->k และ k->j หรือไม่
                if (d[i][k] != INF && d[k][j] != INF) {
                    // อัปเดตระยะทางที่สั้นที่สุด [cite: 176]
                    if (d[i][k] + d[k][j] < d[i][j]) {
                        d[i][j] = d[i][k] + d[k][j];
                    }
                }
            }
        }
    }
}
```

Optimal Binary Search tree $C(i,j) = \min(C(i,m-1) + C(m+1,j)) + \sum p_k$

word	A	B	C	D
prob	0.3	0.15	0.5	0.05

$\sum p_k$	1	2	3	4
1	0.3	0.45	0.95	1.00
2		0.15	0.65	0.70
3			0.5	0.55
4				0.05

$C(i,j)$	1	2	3	4
1	1	1	3	3
2		2	3	3
3			3	3
4				4



OBST cost = $(0.5 \times 1) + (2 \times (0.3 + 0.05)) + (3 \times 0.15) = 0.5 + 0.7 + 0.45$

$L=1 \text{ node}, L=2 \text{ node}, L=3, L=4, L=k$

$C(i,j) = W(i,j) + \min\{C(i,k-1) + C(k+1,j)\}$

ex. $C(1,2) = W(1,2) = 0.45$

$k=1: C(1,0) + C(2,2) = 0 + 0.15 = 0.15$
 $k=2: C(1,1) + C(3,2) = 0.3 + 0 = 0.3$

$\therefore C(1,2) = 0.45 + 0.15 = 0.6, \therefore C(1,2) = 1 = k$

```
// p[]: array ของความน่าจะเป็น (probabilities) ของ n
// n: จำนวนโหนดในกราฟ

void OptimalBST_DP(double p[], int n) {
    // 1. เริ่มต้นตาราง: c เก็บค่า, r เก็บราก, pp เก็บค่าความน่าจะเป็นสะสม
    double c[n+2][n+2];
    int r[n+1][n+1];
    double pp[n+1];

    // 2. คำนวณค่าความน่าจะเป็นสะสม pp[i, j]
    for (int i = 1; i <= n; i++) {
        pp[i] = p[i-1];
        for (int j = i+1; j <= n; j++) {
            pp[j] = pp[j-1] + p[j-1];
        }
    }

    // 3. กำหนดค่าเริ่มต้น (Base Cases)
    for (int i = 1; i <= n; i++) {
        c[i][i-1] = 0; // กรณีไม่มีโหนด
    }

    // 4. เริ่ม Loop ตามความยาวของช่วง k (Chain Length)
    for (int k = 1; k <= n; k++) {
        // 4.1. คำนวณค่าความน่าจะเป็นสะสม [cite: 394]
        for (int i = 1; i <= n - k + 1; i++) {
            int j = i + k - 1;
            c[i][j] = 1e9; // กำหนดค่าเริ่มต้น Infinity

            // 5. หาจุดเชื่อมต่อใน ช่วง i ถึง j หาจุดใน (Root)
            for (int m = i; m <= j; m++) {
                double newCost = c[i][m-1] + c[m+1][j] + pp[j]-pp[i-1];

                if (newCost < c[i][j]) {
                    c[i][j] = newCost;
                    r[i][j] = m; // บันทึกการเชื่อมต่อ
                }
            }
        }
    }
}
```

Greedy Algorithm

Fractional Knapsack

ของ	W	B	① $V_i = b_i/w_i$	② sort by V	③ Select
1	10	60	$V_1 = 60/10 = 6$	1 V_1	$V_1: 10 = 60$
2	20	100	$V_2 = 100/20 = 5$	2 V_2	$V_2: 20 = 100$
3	30	120	$V_3 = 120/30 = 4$	3 V_3	$V_3: 20 = 80$

```
// โครงสร้างข้อมูลสำหรับสิ่งของ
struct Item {
    int value;
    int weight;
    double density;
    // ถ้า w_i = value / weight
};

// ฟังก์ชันสำหรับเปรียบเทียบสิ่งของเพื่อเรียงลำดับจากมากไปน้อย
bool compare(Item a, Item b) {
    return a.density > b.density;
}

double fractionalKnapsack(int W, vector<Item>& items, int n) {
    // 1. คำนวณค่า density สำหรับทุกสิ่งของ
    for (int i = 0; i < n; i++) {
        items[i].density = (double)items[i].value / items[i].weight;
    }

    // 2. เรียงลำดับของสิ่งของตามค่า density (Sort by density)
    sort(items.begin(), items.end(), compare);

    double totalValue = 0.0; // มูลค่ารวมของสิ่งของทั้งหมด
    int currentWeight = 0; // น้ำหนักปัจจุบันในกระเป๋า

    // 3. เริ่มเลือกสิ่งของตามลำดับความหนาแน่น
    for (int i = 0; i < n; i++) {
        // ถ้าสิ่งของนี้หนักกว่ากระเป๋าที่เหลือ
        if (currentWeight + items[i].weight <= W) {
            currentWeight += items[i].weight;
            totalValue += items[i].value;
        } else {
            // ถ้าสิ่งของนี้หนักเกินไป ให้ "แบ่งของ" เฉพาะส่วนที่พอดี
            int remainingCapacity = W - currentWeight;
            totalValue += items[i].value * ((double)remainingCapacity / items[i].weight);
            break; // เป็นอันจบ การแบ่งของ
        }
    }

    return totalValue;
}
```

Activity Selection

step: sort $\rightarrow$ stop time (f)

Pick $\rightarrow$ stop latest

choice: $s_i \text{ ใหญ่ } > f_i$ ถ้า

รับเข้าจากกิจกรรม

```
T(n) = O(n log n)

// โครงสร้างข้อมูลสำหรับกิจกรรม
struct Activity {
    int id; // ตัวระบุสำหรับกิจกรรม
    int start; // เวลาเริ่มต้น (s)
    int finish; // เวลาสิ้นสุด (f)
};

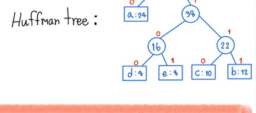
// ฟังก์ชันสำหรับเปรียบเทียบกิจกรรมเพื่อเรียงลำดับตามเวลาสิ้นสุด (f) จากน้อยไปมาก
bool compareActivity(Activity a, Activity b) {
    return a.finish < b.finish;
}

void greedyActivitySelect(vector<Activity>& activities, int n) {
    // 1. เรียงลำดับกิจกรรมตามเวลาสิ้นสุด (Finish time)
    sort(activities.begin(), activities.end(), compareActivity);

    // 2. เลือกกิจกรรมแรกเสมอ (เพราะมันเร็วที่สุด)
    cout << "Selected activities: ";
    int lastFinishTime = activities[0].finish;
    cout << activities[0].id << " ";

    // 3. วนลูปพิจารณาเลือกกิจกรรมที่เหลือ
    for (int i = 1; i < n; i++) {
        // ถ้าเวลาเริ่มต้นของกิจกรรมปัจจุบันมากกว่าหรือเท่ากับเวลาสิ้นสุดของกิจกรรมก่อนหน้า
        if (activities[i].start >= lastFinishTime) {
            cout << activities[i].id << " ";
            lastFinishTime = activities[i].finish; // อัปเดตเวลาสิ้นสุด
        }
    }
}
```

Huffman code : a: 0, b: 11, c: 10, d: 100, e: 101



String Matching

Boyer-Moore (right-left)

Bad-symbol: $d = \max(1, tcc - k)$

ไม่ชนใน pattern

Text String matching

Pattern match

$k=0, tcm=5$
 $d = \max(1, 5-0) = 5$

$k=0, tcc=2$
 $d = \max(1, 2-0) = 2$

* ตัวที่ไม่ใช่ pattern

Good-suffix

ex. k pattern d

1 ASACAG 4

2 ASACAG 4

3 ASACAG 4

4 ASACAG 4

5 ASACAG 4

Knuth-Morris-Pratt (left-right)

ex. AGACAAGA

prefix suffix

AGACA

การคำนวณ shift

1) ถ้า pattern ไม่ชนในหลักที่ k

AGCCA

AGCGT

AGAGT

AGAGT

AGGAGA

AGGAGT

AGGAGT

AGGAGT

```
// KMP
void KMPSearch(string T, string P) {
    int n = T.length();
    int m = P.length();
    int pi[m];

    computePrefixFunction(P, pi);

    int q = 0; // จำนวนตัวอักษรที่ match ได้ในขณะนี้
    for (int i = 0; i < n; i++) {
        // เมื่อเกิด Mismatch: ใช้ตาราง pi เพื่อเลื่อน pattern
        while (q > 0 && P[q] != T[i]) {
            q = pi[q-1];
        }

        // เมื่อเกิด Match: เพิ่มจำนวนตัวอักษรที่ match
        if (P[q] == T[i]) {
            q++;
        }

        // เมื่อ Pattern หมดตัว
        if (q == m) {
            cout << "Wu Match ที่ index: " << (i - m + 1) << " ";
            q = pi[q-1]; // เริ่มนับ match ใหม่
        }
    }
}
```

Summary

ok (m, match) $\rightarrow$ k $\rightarrow$ shift

```
// โครงสร้างโหนดของ Huffman Tree
struct Node {
    char data; // ตัวอักษร
    int freq; // ความถี่ [cite: 657]
    Node *left, *right;
};

// ฟังก์ชันสำหรับเปรียบเทียบสำหรับ Priority Queue (Min-Heap)
struct compare {
    bool operator()(Node* l, Node* r) {
        return l->freq > r->freq; // บวกค่าความถี่
    }
};

Node* buildHuffmanTree(char data[], int freq[], int n) {
    // 1. สร้างโหนดใน Min-Heap
    priority_queue<Node*, vector<Node*>, compare> minHeap;
    for (int i = 0; i < n; i++) {
        minHeap.push(new Node(data[i], freq[i]));
    }

    // 2. วนลูปจนเหลือโหนดเดียวใน Heap
    while (minHeap.size() != 1) {
        // ดึง 2 โหนดออกมาเพื่อเชื่อมต่อกัน (L, R)
        Node *left = minHeap.top(); minHeap.pop();
        Node *right = minHeap.top(); minHeap.pop();

        // 3. สร้างโหนดใหม่เป็นโหนดพ่อแม่ของสองโหนดที่ดึงออกมา
        // 's' คือสัญลักษณ์สำหรับโหนดพ่อแม่ (Internal Node)
        Node *parent = new Node('s', left->freq + right->freq);
        parent->left = left;
        parent->right = right;

        // 4. เพิ่มโหนดพ่อแม่กลับเข้าไปใน Heap
        minHeap.push(parent);
    }

    // 5. คืนค่ารากของ Huffman Tree
    return minHeap.top();
}

// ฟังก์ชันสำหรับแสดงรหัส (จากซ้ายไปขวา)
void printCodes(Node* root, string str) {
    if (!root) return;

    if (root->data != 's') { // ถ้าเป็นโหนด (Leaf)
        cout << root->data << " ";
        if (str << endl;

        printCodes(root->left, str + "0"); // ด้านซ้าย 0
        printCodes(root->right, str + "1"); // ด้านขวา 1
    }
}
```

```
void BoyerMooreSearch(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();
    int d2[m+1]; // array ของ Bad-symbol shift
    int d2[m+1]; // array ของ Good-suffix shift

    // 1. Preprocessing: คำนวณ d2 และ d2
    preprocessBadSymbol(pattern, d2);
    preprocessGoodSuffix(pattern, d2);

    // 2. Matching: เริ่มนับจำนวนตัวอักษรที่ match
    while (1) {
        int i = 0; // ดัชนีของ pattern
        while (i < m) {
            if (text[i+m-d2[i]] != pattern[i]) {
                i++;
                continue;
            }
            i++;
        }
        // เมื่อพบการ match
        if (i == m) {
            cout << "Wu Match ที่ index: " << (i - m + 1) << " ";
            i = i - m + 1; // เริ่มนับ match ใหม่
        }
    }
}
```